

CS3.301: Operating Systems and Networks

Lecture 4: Limited Direct Execution

INSTRUCTOR: DR. DEEPAK GANGADHARAN

Direct Execution

OS	Program
1. Create an entry in process list 2. Allocate memory for the program 3. Load program into memory 4. Setup stack with argc/argv 5. Clear registers 6. Execute call main()	
	7. Run main () 8. Execute return from main()
9. Free memory of process 10. Remove process from process list	

Limited Direct Execution

- Low level mechanism with a separate user space and kernel space
- Let the program run directly on the CPU
- Limits what a process can do
- Offer privileged operations through a well-defined interface facilitated by the OS

OS is more than just a library!

Limited Direct Execution (contd...)

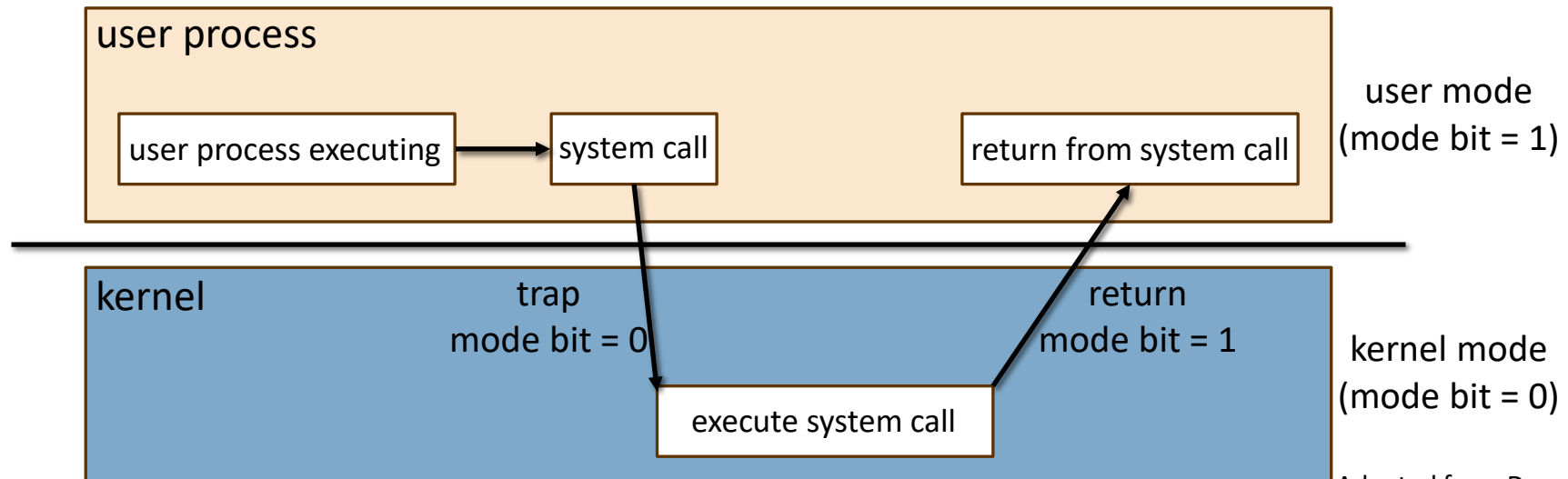
- Introduce a new processor mode – **user mode**.
- Code that runs in user mode restricted.
- When running in user mode, a process can't issue I/O requests.
- If done → processor raises exception and OS would kill the process.
- OS runs in **kernel mode** and code in kernel mode can do what it likes.

Limited Direct Execution (contd...)

- When a user process wishes to perform a privileged operation
 - a special **TRAP** instruction is invoked
 - TRAP saves the caller's registers
 - TRAP jumps into the kernel and raises the privilege mode to **kernel mode**
 - User code specifies a system call number
 - **TRAP table** has the jump address for the system call number
- When finished, the OS calls a special **return-from-trap** instruction, which results in calling user program while simultaneously reducing privilege level back to user mode

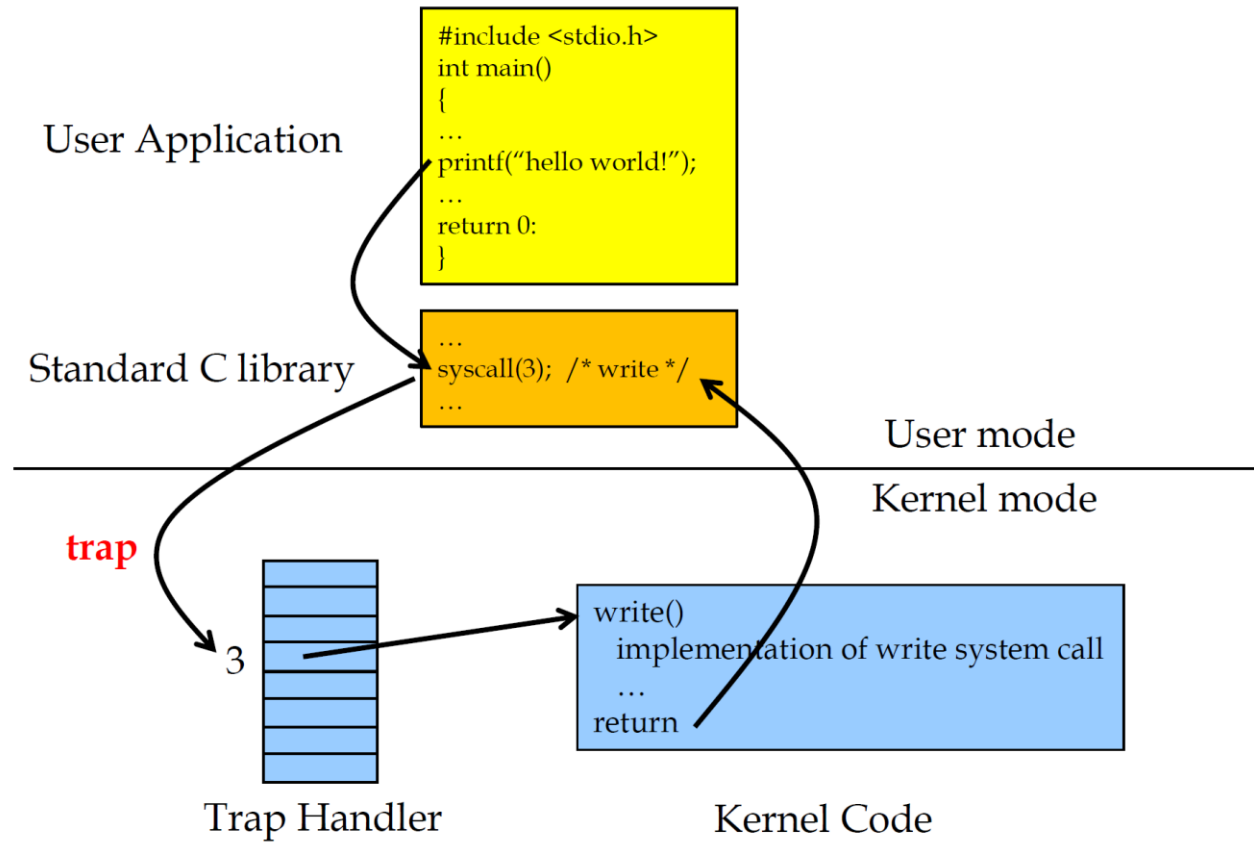
Dual Modes

- Dual-mode operation allows OS to protect itself and other system components
 - User mode and kernel mode (supervisor mode, privileged mode)
 - Mode bit provided by hardware
- Mode change
 - When a trap or interrupt occurs, the hardware switches from user to kernel mode



Adapted from Dongkun Shin's lecture on LDE

Dual Modes



Interrupt and Trap

➤ Interrupt

- Unexpected external event
- I/O Interrupt, Clock Interrupt, Console Interrupt
- From either the hardware or software
 - Hardware may trigger an interrupt by sending a signal to the CPU

➤ Trap

- Software generated interrupt caused by
 - Exception: An error from running a program (div by zero)
 - System call: A specific request from a user program

Limited Direct Execution

OS @ boot (Kernel mode)	Hardware	
Initialize trap table	remember address of ... syscall handler	
OS@run (Kernel mode)	Hardware	Program (User mode)
Create entry for process list Allocate memory for program Load program into memory Setup user stack with argv Fill kernel stack with reg/PC return-from-trap	restore regs (from kernel stack) move to user mode jump to main	
		Run main() ... Call system call trap into OS

Limited Direct Execution

OS@run (Kernel mode)	Hardware	Program (User mode)
	 save regs (to kernel stack) move to kernel mode jump to trap handler	
Handle trap Do work of syscall return-from-trap		
	restore regs (from kernel stack) move to user mode jump to PC after trap	
		 return from main() trap (via exit())
Free memory of process Remove process from process list		

Problem 2: Switching between Processes

- How can the OS regain control of the CPU so that it can switch between processes?
- **A Cooperative Approach: Wait for System Calls**
 - OS trusts the processes of the system to behave reasonably.
 - Processes that run too long are assumed to periodically give up the CPU so that the OS can decide to run some other task.
 - Using explicit **yield** system call
 - Applications transfer control to OS when they do something illegal (divide by zero or attempt to access memory it shouldn't) by generating a trap to OS → OS will likely terminate the process
 - Passive Approach: What will happen if a process (malicious or full of bugs) ends up in an infinite loop and never makes a system call?

Problem 2: Switching between Processes

➤ A Non-Cooperative Approach: The OS takes Control

- Cooperative Approach: When a process gets stuck in an infinite loop, the only way to return control to OS is by resorting to **reboot the machine**.
- Solution: **timer interrupt** – A timer device can be programmed to raise an interrupt every T milliseconds
- Interrupt raised → Currently running process halted → A pre-configured interrupt handler in the OS runs.
- OS regains control of CPU
- Two possibilities – Continue with current process after interrupt handler is executed or stop the current process and switch to a different process.
- During boot sequence, OS starts the timer.

Saving and Restoring Context

- Switching decisions made by a part of the OS called scheduler.
- Decision made to switch → OS executes a low-level piece of code referred to as **context switch**
- Save a few register values such as general purpose registers, PC and kernel stack pointer for the currently executing process (onto its kernel stack)
- Restore a few for the soon to be executing process (from its kernel stack)
- return-from-trap execution → system resumes execution of new process

LDE Protocol (Timer Interrupt)

OS @ boot (Kernel mode)	Hardware	
Initialize trap table	remember address of ... syscall handler Timer handler	
Start Interrupt timer	Start timer Interrupt CPU every "T" millisecond	
OS@run (Kernel mode)	Hardware	Program (User mode)
		Process A
	Timer Interrupt Save regs(A) to kernel stack (A) Move to kernel mode Jump to trap handler	

LDE Protocol (Timer Interrupt)

OS@run (Kernel mode)	Hardware	Program (User mode)
Handle the trap Call switch() routine Save regs(A) to proc-struct(A) Restore regs(B) from proc-struct(B) Switch to kernel stack (B) Return-from-trap (into B)		
	 Restore regs (B) from kernel stack (B) move to user mode jump to B's PC	
		Process B

Multiple Interrupts

- Interrupt during trap handling - What happens if during a system call, a timer interrupt occurs?
- Interrupt during interrupt handling
- One simple solution – disable interrupts during interrupt processing → **What happens if you disable for too long?**
- Sophisticated locking mechanism to protect concurrent access to internal data structures

Thank You!!